

Report: Assignment 1 -Builder Pattern (CarBuilder)

by Abraimov Yeskendir SE-2502

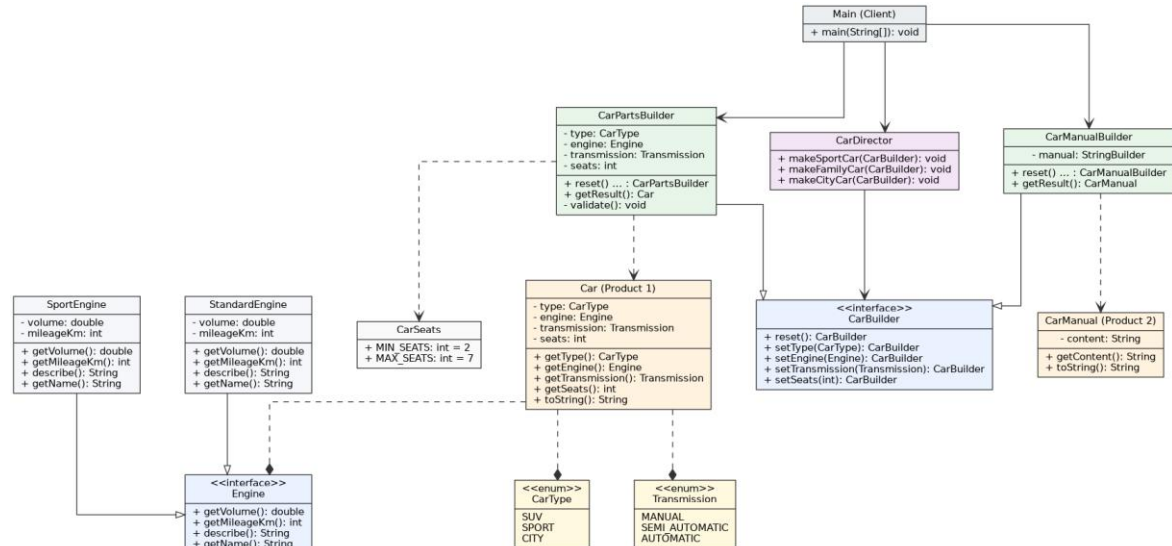
1. Introduction

For this assignment I built a small car assembly system, CarBuilder. A car is a good fit for the Builder pattern because it's made of several parts that can vary independently: the body type (SUV, sport, or city), the engine (sport or standard, each with its own volume and mileage), the transmission, and the number of seats.

Without Builder, I'd either need one constructor with a long list of parameters, or a separate constructor for every combination of options. That gets error-prone fast -it's easy to mix up two arguments of the same type, like seats and mileage, since both are just numbers. Builder solves this by letting me set each part by name, one at a time, and only assembling the final object once everything is in place.

The part I found most useful about this pattern is that the same sequence of construction steps can produce two completely different results. A single CarDirector recipe, like makeSportCar, can be handed either a CarPartsBuilder (which produces a real Car object) or a CarManualBuilder (which produces a plain-text description of that same car). Neither the recipe nor the Director needs to change - only the builder passed in does.

2. UML Class Diagram



3. Clean Code Principles Applied

1. Meaningful Names

I tried to apply that to methods and fields throughout the project - `makeFamilyCar` tells you what car it builds, `getMileageKm` tells you both what it returns and its unit, and

CarDirector.java -method names double as documentation

MIN_SEATS/MAX_SEATS tell you the rule without needing a comment next to them.

```
public void makeFamilyCar(CarBuilder builder) {
    builder.reset()
        .setType(CarType.SUV)
        .setEngine(new StandardEngine(3.0, 30000))
        .setTransmission(Transmission.AUTOMATIC)
        .setSeats(5);
}
```

2. Classes Should Be Small

Chapter 10 argues that a class should have one reason to change, and that a small, focused set of classes is easier to understand than one class doing several jobs. In this project, CarDirector only knows the order and values for a recipe, not how a Car actually gets created; CarPartsBuilder only collects parts and validates them; and Car itself has no logic at all - it's just a holder for the finished result.

Car.java -no behavior beyond holding and exposing its own data

```
public final class Car {
    private final CarType type;
    private final Engine engine;
    private final Transmission transmission;
    private final int seats;

    Car(CarType type, Engine engine, Transmission transmission, int seats) {
        this.type = type;
        this.engine = engine;
        this.transmission = transmission;
        this.seats = seats;
    }
}
```

3. Small functions that do one thing

Martin's rule for functions in Chapter 3 is that they should be short, and that everything in a function should be working at the same level of detail toward one single task. Every setter in CarPartsBuilder and CarManualBuilder is one line - it assigns a field or appends text, nothing else. Validation is pulled out into its own method instead of being mixed into the setters, so setType() stays doing exactly one thing: setting the type.

CarPartsBuilder.java

```
@Override
public CarPartsBuilder setType(CarType type) {
    this.type = type;
    return this;
}
```

4. Few arguments, each with one job

This section argues that functions with many parameters are hard to read and easy to call incorrectly, and that a single, well-named argument (a “monadic” function) is the easiest case to understand. Rather than one Car constructor taking type, engine, transmission and seats all at once, CarBuilder exposes four separate one-argument setters - each call makes it obvious, by name, exactly which part of the car is being set.

CarBuilder.java -one argument per step instead of one large parameter list

```
public interface CarBuilder {
    CarBuilder reset();
    CarBuilder setType(CarType type);
    CarBuilder setEngine(Engine engine);
    CarBuilder setTransmission(Transmission transmission);
}
```

```
    CarBuilder setSeats(int seats);  
}
```

5. Prefer exceptions to returning error codes

Martin points out that returning an error code forces the caller to check it immediately, and it's easy to forget that check; throwing an exception separates the error-handling path from the main logic and can't be silently ignored. `CarPartsBuilder.getResult()` follows this directly: instead of returning null or a boolean “success” flag when the configuration is incomplete, it throws an `IllegalStateException` with a message describing exactly what's wrong.

CarPartsBuilder.java

```
private void validate() {  
    if (type == null) {  
        throw new IllegalStateException(  
            "Car type must be set before building a car");  
    }  
    if (engine == null) {  
        throw new IllegalStateException(  
            "Engine must be set before building a car");  
    }  
    if (transmission == null) {  
        throw new IllegalStateException(  
            "Transmission must be set before building a car");  
    }  
    if (seats < CarSeats.MIN_SEATS || seats > CarSeats.MAX_SEATS) {  
        throw new IllegalStateException(  
            "Seats must be between %d and %d, but was %d"  
            .formatted(CarSeats.MIN_SEATS, CarSeats.MAX_SEATS, seats));  
    }  
}
```

6. Don't Repeat Yourself

Martin treats duplication as one of the main things worth eliminating from a codebase, since the same logic (or the same set of values) repeated in two places means two places to update, and two chances to get it wrong. `CarDirector` exists specifically to avoid repeating a car's configuration everywhere it's needed: `makeSportCar` is written once and reused for both a `CarPartsBuilder` and a `CarManualBuilder`, instead of the same type/engine/transmission/seats values being typed out twice.

CarDirector.java -one recipe, reused by two different builders

```
public void makeSportCar(CarBuilder builder) {  
    builder.reset()  
        .setType(CarType.SPORT)  
        .setEngine(new SportEngine(8.0, 5000))  
        .setTransmission(Transmission.MANUAL)  
        .setSeats(2);  
}
```

7. Replace magic numbers with named constants

This heuristic from Martin's “Smells and Heuristics” chapter says that raw numbers in the middle of logic hide their own meaning - a reader has to guess what 2 and 7 refer to. The valid seat range is defined once, by name, in `CarSeats`, and the validation check reads off those names instead of repeating the numbers themselves.

CarSeats.java

```
public final class CarSeats {  
    public static final int MIN_SEATS = 2;  
    public static final int MAX_SEATS = 7;  
  
    private CarSeats() {  
    }  
}
```

4. Conclusion

Building this project made the benefits of Builder feel concrete rather than theoretical. Centralizing all the validation in one place (`getResult()`) meant I never had to worry about a half-configured Car slipping through - something that's genuinely awkward to guarantee with a plain constructor once you have more than two or three required fields.

The biggest win was reusing the exact same recipe for two unrelated outputs. Writing `makeSportCar` once and getting both a working Car object and a readable text manual out of it, just by swapping which builder I passed in, is what made the pattern click for me -it's not just about avoiding long constructors, it's about separating "what steps to follow" from "what the steps actually produce".

5. GitHub Repository

Repository: <https://github.com/Naitskaut/CarBuilder>